# SuperMem MVP 心跳任务 - 启动与配置指南

## 前置条件

在启动心跳任务之前，确保你已经：

1. 成功部署了 SuperMem 生产库（默认端口 8764）
2. 配置了 LLM 代理（可选，推荐端口 8777）
3. 有一个可以运行定时任务的环境（cron、systemd timer、或任何定时调度系统）

---

## 心跳任务的核心逻辑

```
每隔10分钟执行一次：
    1. 获取最近10分钟的对话内容
    2. 过滤掉工具日志、系统消息
    3. 调用 SuperMem /api/extract 接口
    4. 自动去重，写入记忆库
```

---

## 方案一：Bash 脚本 + Cron（最简单）

这是最通用的方案，任何 Linux 环境都可以用。

### Step 1: 创建心跳脚本

创建 `supermem-heartbeat.sh`：

```
#!/bin/bash

# 配置
SUPERMEM_URL="http://127.0.0.1:8764"
LLM_PROXY_URL="http://127.0.0.1:8777"   # 可选，没有就注释掉
LOG_FILE="./heartbeat.log"

# 获取最近10分钟的对话内容
# 注意：这里需要你自己实现"获取对话内容"的逻辑
```

```
# 不同平台（飞书、微信、Discord）的对话获取方式不一样
# 这里提供一个通用的模板，你需要根据实际情况修改

get_recent_conversations() {
    # 示例：从你的平台获取对话（需要替换成你的实际API）
    # curl -s "https://你的平台API/conversations?limit=10" \
    #     -H "Authorization: Bearer YOUR_TOKEN"

    # 或者：从本地文件读取最近的对话
    # tail -n 100 ./conversations.log

    # ... 实现你的对话获取逻辑 ...

    # 返回格式：纯文本的对话内容
    echo "
用户：你好，我想写一个关于时间回档的小说
AI：好的，这是一个很有意思的题材，你有什么初步的想法吗？
用户：我想让主角回到2012年，发现所有人都不记得一个叫许念的女生
AI：这个设定很有张力，我们可以从细节入手...
"
}

# 过滤对话内容 - 只保留真正的对话，去掉工具日志
filter_conversation() {
    local content="$1"

    # 去掉工具调用相关的行
    content=$(echo "$content" | grep -v "工具调用" | grep -v "tool_call" | grep
-v "执行结果")

    # 去掉系统提示词相关的行
    content=$(echo "$content" | grep -v "system_hint" | grep -v "系统提示")

    # 去掉心跳任务自己的报告
    content=$(echo "$content" | grep -v "心跳任务" | grep -v "heartbeat")

    echo "$content"
}

# 提取到SuperMem
extract_to_supermem() {
    local content="$1"

    # 检查内容是否为空
    if [ -z "$content" ] || [ "$content" = "" ]; then
        echo "$(date '+%Y-%m-%d %H:%M:%S') - 无新对话，跳过提取" >> "$LOG_FILE"
```

```bash
        return 0
    fi

    # 调用提取接口
    response=$(curl -s -X POST "$SUPERMEM_URL/api/extract" \
        -H "Content-Type: application/json" \
        -d "{\"content\": $(echo "$content" | jq -R -s '.')}")

    # 记录结果
    nodes_added=$(echo "$response" | jq -r '.nodes_added // 0')
    edges_added=$(echo "$response" | jq -r '.edges_added // 0')

    echo "$(date '+%Y-%m-%d %H:%M:%S') - 提取完成：新增 $nodes_added 节点，
$edges_added 边" >> "$LOG_FILE"
}

# 主流程
echo "========================================" >> "$LOG_FILE"
echo "$(date '+%Y-%m-%d %H:%M:%S') - 心跳任务开始" >> "$LOG_FILE"

# 1. 获取对话
raw_conversation=$(get_recent_conversations)

# 2. 过滤对话
filtered_conversation=$(filter_conversation "$raw_conversation")

# 3. 提取到SuperMem
extract_to_supermem "$filtered_conversation"

echo "$(date '+%Y-%m-%d %H:%M:%S') - 心跳任务完成" >> "$LOG_FILE"
echo "========================================" >> "$LOG_FILE"
```

### Step 2: 给脚本加执行权限

```bash
chmod +x supermem-heartbeat.sh
```

### Step 3: 配置 Cron 定时任务

编辑 crontab：

```bash
crontab -e
```

添加以下内容（每10分钟执行一次）：

```bash
*/10 * * * * cd /path/to/your/supermem && ./supermem-heartbeat.sh
```

### Step 4: 验证

查看日志文件，确认心跳任务正常执行：

```bash
tail -f heartbeat.log
```

---

## 方案二：Python 脚本（更灵活）

如果你需要更复杂的逻辑，用 Python 会更方便。

### supermem-heartbeat.py

```python
import requests
import time
import json
from datetime import datetime, timedelta
import os

# 配置
SUPERMEM_URL = os.getenv("SUPERMEM_URL", "http://127.0.0.1:8764")
LLM_PROXY_URL = os.getenv("LLM_PROXY_URL", "http://127.0.0.1:8777")
LOG_FILE = "./heartbeat.log"
EXTRACT_INTERVAL = 10  # 分钟

def log(message):
    """写日志"""
    timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
```

```
        with open(LOG_FILE, 'a', encoding='utf-8') as f:
            f.write(f"[{timestamp}] {message}\n")
        print(f"[{timestamp}] {message}")

def get_recent_conversations(minutes=10):
    """
    获取最近N分钟的对话内容
    【你需要根据你的实际平台修改这个函数】

    支持的平台示例:
    - 飞书: 调用飞书开放平台 API
    - 微信: 通过微信机器人API获取
    - 本地: 从日志文件读取
    """
        # TODO: 替换成你的对话获取逻辑
        # 示例：从你的平台API获取
        # headers = {"Authorization": f"Bearer {os.getenv('YOUR_TOKEN')}"}
        # response = requests.get(
        #     "https://你的平台API地址/conversations",
        #     headers=headers,
        #     params={"limit": 50}
        # )
        # return parse_conversations(response.json())

        #【重要】这是示例代码，你必须替换成你自己的对话获取方式

        return """
用户：你好，我想写一个关于时间回档的小说
AI：好的，这是一个很有意思的题材，你有什么初步的想法吗？
用户：我想让主角回到2012年，发现所有人都不记得一个叫许念的女生
AI：这个设定很有张力，我们可以从细节入手...
    """

def filter_conversation(content):
    """过滤对话内容，只保留真正的对话"""
    if not content or not content.strip():
        return ""

    lines = content.split('\n')
    filtered = []

    for line in lines:
        line = line.strip()
        if not line:
            continue
```

```
            # 跳过工具日志、系统消息、心跳报告
            skip_keywords = [
                "工具调用", "tool_call", "执行结果",
                "system_hint", "系统提示",
                "心跳任务", "heartbeat",
                "日程任务", "calendar"
            ]

            if any(keyword in line for keyword in skip_keywords):
                continue

            filtered.append(line)

        return '\n'.join(filtered)

def extract_to_supermem(content):
    """提取对话内容到SuperMem"""
    if not content or not content.strip():
        log("无新对话内容，跳过提取")
        return

    try:
        response = requests.post(
            f"{SUPERMEM_URL}/api/extract",
            json={"content": content},
            timeout=60
        )

        if response.status_code == 200:
            result = response.json()
            nodes_added = result.get("nodes_added", 0)
            edges_added = result.get("edges_added", 0)
            log(f"提取成功：新增 {nodes_added} 节点，{edges_added} 边")
        else:
            log(f"提取失败：HTTP {response.status_code} - {response.text}")

    except Exception as e:
        log(f"提取异常：{str(e)}")

def main():
    """主流程"""
    log("=" * 50)
    log("心跳任务开始执行")

    # 1. 获取最近10分钟的对话
    content = get_recent_conversations(EXTRACT_INTERVAL)
```

```python
        # 2. 过滤内容
        filtered = filter_conversation(content)

        # 3. 提取到SuperMem
        extract_to_supermem(filtered)

        log("心跳任务执行完成")
        log("=" * 50)

if __name__ == "__main__":
    main()
```

### 运行方式

```bash
# 直接执行一次测试
python3 supermem-heartbeat.py

# 用 cron 定时执行（每10分钟一次）
*/10 * * * * cd /path/to/supermem && python3 supermem-heartbeat.py
```

---

## 关键配置项

| 配置项 | 推荐值 | 说明 |
| --- | --- | --- |
| 执行频率 | 每10分钟 | 不要太频繁（<5分钟），会增加服务器负担；也不要太稀疏（>30分钟），会丢失对话 |
| 去重阈值 | 0.85 | 向量相似度高于这个值的内容会被认为重复，跳过提取 |
| 内容过滤 | 开启 | 必须过滤工具日志、系统消息，否则记忆库会很乱 |
| 日志记录 | 开启 | 方便排查问题，日志不要超过100MB，记得轮转 |

---

## 常见问题

### Q: 心跳任务重复提取同一段对话怎么办？

A: SuperMem 的 `/api/extract` 接口内置了自动去重。相同或高度相似的内容只会被提取一次。你不需要额外处理。

### Q: 为什么心跳任务不提取工具日志？

A: 工具日志是执行过程的副产品，不是真正的"对话内容"。提取它们只会让记忆库变得混乱，没有实际价值。

### Q: 心跳任务会消耗我的LLM额度吗？

A: 不会。提取功能走的是本地代理通道，不消耗你的 LLM 额度。

### Q: 我可以修改执行频率吗？

A: 可以。推荐 10-15 分钟一次。不要低于 5 分钟，也不要高于 30 分钟。

### Q: 心跳任务失败了怎么办？

A: 检查日志文件，通常是：

1. SuperMem 服务没启动
2. 对话获取API配置错了
3. 网络不通

---

## 健康检查

建议添加一个简单的健康检查，确保心跳任务正常运行：

```
#!/bin/bash
# heartbeat-health.sh

LOG_FILE="./heartbeat.log"

# 检查最近30分钟内是否有成功执行的记录
last_success=$(tail -n 100 "$LOG_FILE" | grep "提取成功" | tail -n 1)
```

```bash
if [ -z "$last_success" ]; then
    echo "警告：心跳任务最近30分钟没有成功执行！"
    # 可以在这里加报警逻辑，比如发邮件或飞书消息
fi
```

---

## 总结

心跳任务的本质很简单：**定期把对话内容喂给记忆系统**。

真正的难点不在于代码，而在于"获取对话内容"——不同的平台（Coze、飞书、微信）有不同的API，你需要根据你的实际环境来实现 `get_recent_conversations` 这个函数。

只要你能拿到对话文本，剩下的就都交给 SuperMem 了。